

Structuri de date fundamentale

CAPITOLUL II

Cuprins

Introducere

Tipuri de date

Tipuri de date abstracte

Tipuri nestructurate

Tipuri structurate

Exerciții

Introducere

Cantitatea mare de informații pe care o prelucrează un sistem de calcul reprezintă o **abstractizare** a lumii înconjurătoare

Informația – o mulțime de date selectate referitoare la lumea reală

Datele selectate - mulțimea de date considerată cea mai reprezentativă pentru problema tratată, se presupune că din ea pot fi deduse rezultatele dorite

Introducere

Date – **abstractizare** a realității

Abstractizare = o simplificare

Alegerea unei abstractizări convenabile:

1. Definirea unui set **reprezentativ** de date care modelează situația reală
2. Stabilirea **reprezentării** abstractizării

Tipuri de date

Un **tip de date** este o **colecție de valori** la care se asociază o **serie de operații**

Ex: tipul boolean:

- Valori: fals, adevarat
- Operații: negare, ȘI logic, SAU logic, etc.

Trebuie făcută o diferență între **conceptul** logic de tip de date (tipul de date abstract) și **implementarea** acestuia

Ex. O listă este un concept logic

Lista poate fi implementată atât cu tablouri cât și cu pointeri

Tipuri de date abstracte

Tipul de date abstract (TDA) este un model **matematic** împreună cu totalitatea operațiilor definite pe acest model

La nivel conceptual algoritmi pot fi proiectați în termenii unor TDA

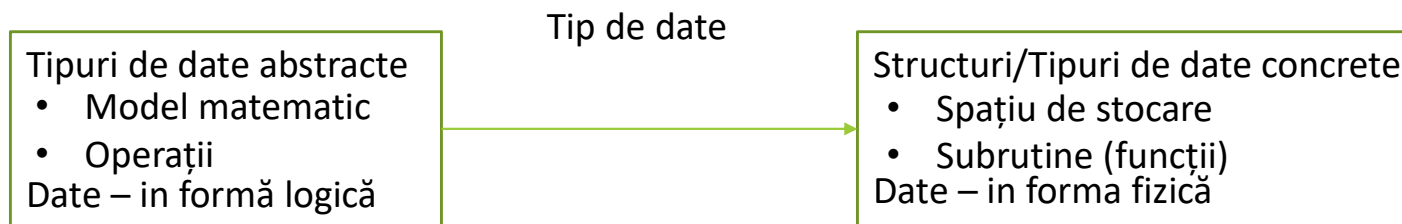
TDA nu conține specificații legate de cum poate fi implementat

Ex. Pentru un automobil avem trei operații de bază: viraj, accelerație, frânare. Două automobile diferite pot implementa operațiile în mod diferit. Cu toate acestea, putem conduce tipuri diferite de mașini, fără să ne intereseze modul de implementare a acestor funcționalități, prin simplul fapt că avem o reprezentare abstractă uniformă a acestora (mașina + operațiile = TDA)

Pentru implementarea într-un limbaj de programare concret trebuie o reprezentare a TDA în termenii **tipurilor de date** și ai operatorilor definiți în acel limbaj

Tipuri de date abstracte

Tipul de date = o implementare a unui TDA



Ex. în C/C++ tipul de date **int** reprezintă o **abstractizare a tipului abstract întreg**

Aceast tip nu reprezintă pe deplin TDA întreg, deoarece are limitări legate de mulțimea de valori care pot fi reținute

Dacă pentru o anumită aplicație aceste limitări nu sunt acceptabile, atunci trebuie folosită o altă implementare pentru numerele întregi (ex. cazul numerelor foarte mari)

Tipuri de date abstracte

Ex. Pentru un **TDA listă** de întregi pot fi definite următoarele operații:

- Inserția unui nou întreg în listă
- Verificare listă vidă
- Numărul de elemente din listă
- Parcurgere listă cu afișare elemente
- Ștergere element

Pentru același TDA pot exista mai multe implementări (ex. cu tablouri, cu pointeri)

Tipuri de date abstracte

Definirea unui **tip de date abstract** presupune

- Precizarea **modelului matematic**
- Definirea **operatorilor** asociați

Conceptul de TDA ne poate ajuta să ne concentrăm pe aspecte cheie, chiar și în probleme non computaționale

Conceptul TDA este foarte important în domeniul calculatoarelor și tehnologiei informației, pentru că ne ajută să abordăm probleme complexe prin abstractizare

Tipuri nestructurate

Tipurile de date pot fi **structurate** (ex. liste, arbori, grafuri) sau **nestructurate** (întregi, reali, etc.).

Tipul **boolean**

TDA boolean are două valori: True (Adevărat) și False (Fals)

Operatorii specifici acestui tip sunt operatorii logici: conjuncție, reuniune și negație

P	Q	P&&Q	P Q	!P
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Tipuri nestructurate

Tipul boolean

- Poate fi implementat folosind tipul enumerare in limbajul C
`typedef enum {False, True} Boolean;`

False = 0

True = 1

Atenție: în cazul operațiilor logice orice valoare diferită de 0 se consideră echivalentă cu valoarea de adevăr True/Adevărat

- In cazul C++ avem un tip predefinit numit bool cu valorile true (1) si false (0)

Tipuri nestructurate

Tipuri **standard predefinite**

- acele tipuri care sunt disponibile în marea majoritate a sistemelor de calcul ca și caracteristici hardware
- pot include numerele întregi, valorile logice, caractere și numere fracționale

Tipuri nestructurate

TDA întreg

- *Modelul matematic: elemente scalare cu valori în mulțimea numerelor întregi $\{\dots;-2;-1;0;1;2;\dots\}$*
- *Notății:*
 - i,j,k – întregi*
 - inz – întreg nonzero*
 - inn – întreg nonnegativ*
 - e – valoare întreaga*
 - b - valoare booleană*

Tipuri nestructurate

Tipul de date (concret) **întreg**

- implementează **o submulțime** a numerelor întregi
- dimensiunea întregilor variază de la un sistem de calcul la altul
- procesul de calcul se întrerupe în cazul obținerii unui rezultat în afara setului reprezentabil (NaN – not a number)
- Pe mulțimea numerelor întregi în afara operatorilor clasici de comparare și atribuire se definesc și operatori standard:

Adunare, scădere, înmulțire, împărțire întreagă și modulo

- Implementările curente ale limbajelor de programare conțin mai multe categorii de tipuri întregi (short, int, long, etc.) care diferă de regulă prin numărul de cifre binare utilizate în reprezentare.

Tipuri nestructurate

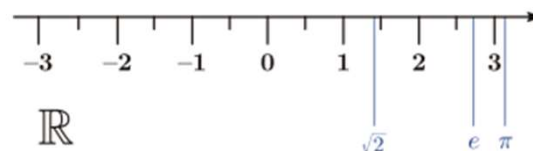
Operații pe TDA întreg:

- *AtribuireIntregi(i,e)* – memorează valoarea lui e în variabila i
- *k = AdunareIntregi(i,j)* (funcție)
- *k = ScădereIntregi(i,j)* (funcție)
- *k = ÎnmulțireIntregi(i,j)* (funcție)
- *k = ÎmpărțireIntregi(i,inz)* (funcție)
- *inn = Modulo(i,inz)* (funcție)
- *b = EgalZero(i)* (funcție)
- *b = MaiMareCaZero(i)* (funcție)
- *b = MaiMicCaZero(i)* (funcție)

Tipuri nestructurate

Tipul de date abstract **real**

- Poate fi văzut ca mulțimea tuturor punctelor de pe o linie, unde punctele corespunzătoare numerelor întregi se află la distanțe egale:



- Exemplu de operații:
- $r = \text{AdunareNrReale}(i,j)$ (funcție)
- $r = \text{ScădereNrReale}(i,j)$ (funcție)
- $r = \text{ÎnmulțireNrReale}(i,j)$ (funcție)
- $r = \text{ÎmpărțireNrReale}(i,j)$ (funcție)
- $r = \text{Radical}(i)$ (funcție)

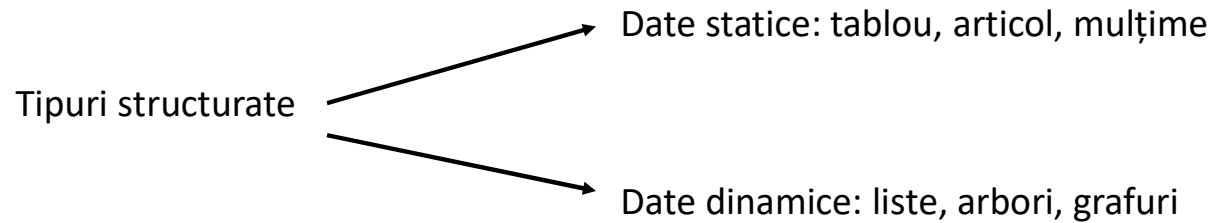
Tipuri nestructurate

Tipul (concret) **real**

- Implementează **o submulțime** a numerelor reale
- Aritmetica numerelor reale este **aproximativă**, în limitele erorilor de rotunjire cauzate de efectuarea calculelor cu un număr finit de cifre zecimale
- Operațiile care conduc la valori care depășesc domeniul de reprezentabilitate al implementării duc la erori
- Implementările curente ale limbajelor de programare conțin mai multe categorii de tipuri reale (float, double, etc.) care diferă de regulă prin dimensiune și precizie.

Tipuri structurate

Tipurile structurate sunt conglomerate de valori componente ale unuia sau mai multe tipuri constitutive definite anterior. Acestea pot fi la rândul lor statice sau dinamice.



Tipuri structurate

TDA Tablou

Model Matematic: Secvență de elemente de același tip. Indicele asociat aparține unui tip ordinal finit. Există o corespondență biunivocă între indici și elementele tabloului.

Notatii

- *tipElement* – tipul elementelor tabloului
- *a* – tablou unidimensional
- *i* – index
- *e* – obiect de tip *tipElement*

Tipuri structurate

Operații pe TDA tablou:

DepuneInTablou(a, i, e) – funcție care depune valoarea lui e în cel de-al i -lea element al tabloului a

$e = \text{FurnizeazăDinTablou}(a, i)$ – funcție care returnează valoarea celui de-al i -lea element al tabloului a

Tipuri structurate

Tipul de date **tablou**

- Un tablou este o structură **omogenă**, el constând dintr-o mulțime de componente de același tip numit tip de bază
- Tabloul este o structură de date cu **acces direct** (random-access), deoarece oricare dintre elementele sale sunt direct și în mod egal accesibile
- Pentru a preciza o componentă individuală, numelui întregii structuri i se asociază un **indice** care selectează pozițional componenta dorită

Tipuri structurate

Tipul concret de date **tablou**

- La definirea unui tablou, se precizează în principiu:
 1. Metoda de structurare, care este implementată direct în limbaj
 2. Numele tipului de date rezultat (tipTablou)
 3. Tipul de bază al tabloului (tipElement)
 4. Tipul indice al tabloului (tipIndice)
 5. Dimensiunea tabloului, adică numărul de elemente

Tipuri structurate

Particularități limbajul C

- Tabloul poate fi implementat ca o structură de date statică pentru care rezervarea de memorie se realizează în faza de compilare a programului.
- Nu se precizează explicit metoda de structurare. Ea este indicată de prezența parantezelor drepte în sintaxa declarației
- Tipul indice este obligatoriu un tip întreg
- Domeniul valorilor indicilor este restrâns la mulțimea întregilor pozitivi
- Între paranteze drepte se indică dimensiunea tabloului (în cazul alocării statice).

Tipuri structurate

```
tipElement numeTablou[nrElemente]; /*definirea unui tablou*/  
  
typedef tipElement tipTablou[nrElemente]; /*definirea unui tip  
tablou*/  
  
tipTablou numeTablou;  
  
/*definirea unui tablou exemple*/  
  
float tablou[10];  
  
char sir[10];  
  
/*definirea unui tip tablou exemplu*/  
  
typedef float tipTablou[100];  
  
tipTablou a, b;
```

Tipuri structurate

Operatori pentru TDA tablou:

Constructorul – crează o instanță a unui tablou

Selectorul – acesta selectează o componentă individuală a unui tablou

```
/*constructia unui tablou*/  
int vect[5] = { 0, 1, 2, 3, 4 };  
int mat[2][3] = { { 0, 1, 2 }, { 3, 4, 5 } };  
char str[18] = "Structuri de date";  
/*selectarea unui element*/  
vect[1]           //index constant  
vect[2*i]         //expresie index
```

Tipuri structurate

Tablouri **multidimensionale**

Accesarea unui element:

```
nume_tablou[indice1][indice2]..[indiceN]
```

Elementele tabloului sunt dispuse în memorie unul după altul, chiar și în cazul tablourilor multidimensionale.

Pentru tabloul:

```
int m[ LIN ] [COL] ;
```

elementul $m[i][j]$ se află pe poziția $i*COL+j$, deci la $i*COL+j$ elemente distanță de începutul tabloului.

Tipuri structurate

```
/* Exemplu de implementare */  
#define numarMaxElem valoareIntreaga  
typedef tipElement tipTablou[numarMaxElem];  
int i;  
tipTablou a;  
tipElement e;  
a[i]=e; //DepuneInTablou(a,I,e)  
e=a[i]; //FurnizeazaDinTablou(a,i)
```

Tipuri structurate

TDA articol

- **Model matematic:** O colecție finită de elemente numite câmpuri, care pot aparține unor tipuri diferite. Există o corespondență biunivocă între lista identificatorilor de câmpuri și colecția de elemente.
- **Notatii**
 - a – obiect de tip articol
 - id – identificador nume câmp
 - e – obiect de același tip cu câmpul id din articolul a

Operatori pentru TDA articol:

- $DepuneArticol(a, id, e)$ – memorează valoarea lui e în câmpul id al lui a
- $e = FurnizeazaArticol(a, id)$ – returnează valoarea câmpului id din a

Tipuri structurate

Structura **articol**

Metoda cea mai generală de a obține tipuri structurate este aceea de a reuni elemente ale mai multor tipuri, unele dintre ele fiind la rândul lor structurate, într-un tip compus

Termenul care descrie o dată compusă de această natură în programare este **struct** (C/C++), respectiv record (Pascal).

Exemple:

- Numerele complexe
- Puncte de coordonate

Tipuri structurate

```
/*definire structura*/  
struct nume_structura{  
    tip_1 listaNumeCamp1;  
    tip_2 listaNumeCamp2;  
    tip_3 listaNumeCamp3;  
    ...  
    tip_n listaNumeCampn;  
} lista_var_structura;
```

Tipuri structurate

```
/*exemple*/  
typedef struct data{  
    int zi, luna, an;  
} data_calendar;  
data_calendar data_nasterii;  
typedef struct student  
{  
    char nume[30], prenume[30];  
    data_calendar data_nasterii;  
    enum stare{admis, respins} situatie; }student;
```

Tipuri structurate

Identificatorii tipului de date articol:

- numeCamp_1...numeCamp_n introduși la definirea tipului sunt nume conferite de către programator componentelor individuale ale tipului structurat
- sunt utilizați ca selectori de articol care permit accesul la câmpurile unei variabile structurate de tip articol
- Exemple
data_calendar d;
d.luna=9;

Tipuri structurate

```
/*exemplu*/  
struct punct  
{  
    int x;  
    int y;};  
struct punct p1;  
double dist;  
dist = sqrt((double) (p1.x*p1.x) + (double) (p1.y*p1.y));
```

Tipuri structurate

Tipul de date **uniune**

O uniune este de fapt o variabilă care poate memora la momente diferite de timp obiecte de tipuri și de dimensiuni diferite. Compilatorul este cel care păstrează evidența și aliniează în mod corespunzător datele memorate

Sintaxa:

- asemănătoare cu cea de la structuri, dar cu cuvântul cheie **union** în față.

Diferența dintre struct și union:

- pentru union lista de câmpuri reprezintă **o listă de variante**, pentru fiecare tip:
 - o variabilă structură conține **toate** câmpurile declarate
 - o variabilă uniune conține **exact una** din variantele declarate.

Dimensiunea unui tip uniune este dată de cel mai mare tip din lista de variante.

Tipuri structurate

```
/*exemplu*/  
union u{  
    int vi;  
    double vr;  
    char *vs; };  
union u valoare;  
valoare.vi=6;           //SAU  
    //valoare.vr=7.5;    //SAU  
//valoare.vs="NAN";
```

Tipuri structurate

Tipul de date abstract **secvență**

- Tipurile structurate prezentate anterior au cardinalitate finită
- Cele mai multe structuri avansate (secvențe, liste, arbori, grafuri, etc.) sunt caracterizate prin cardinalitate infinită

Structura secvență având tipul de date T_0 se definește ca:

$S_0 = \langle \rangle$ (secvența vidă)

$S_i = \langle S_{i-1}, E_i \rangle$, unde $0 < i$ și $E_i \in T_0$

Tipuri structurate

Structura **secvență** este:

- Fie o secvență vidă
- Fie o concatenare a unei secvențe cu un element

Definirea **recursivă** a tipului secvență duce la o cardinalitate infinită

- Fiecare valoare a tipului secvență conține de fapt un număr finit de componente de tip T_0
- Numărul este teoretic nemărginit deoarece, pornind de la orice secvență se poate construi o secvență mai lungă

Tipuri structurate

Consecințe ale cardinalității infinite

- Volumul de memorie necesar reprezentării unei structuri avansate, nu poate fi cunoscut în momentul compilării
- Este necesară aplicarea unor scheme de alocare dinamică a memoriei

Tipuri structurate

TDA **secvență**

- **Modelul matematic:** Secvență de elemente de același tip. Un indicator la secvență indică elementul următor la care se poate realiza accesul. Accesul la elemente este stric secvențial.
- **Notatii**
 - *TipElement* – tipul unui element al secvenței. Nu poate fi de tip secvență
 - *f* – variabilă secvență
 - *e* – variabilă de tip element
 - *b* – valoare booleană
 - *numeFisierDisc* – șir de caractere

Tipuri structurate

TDA secvență

- **Operatori**

- *FurnizeazăSecvență (f,e) – acționează în regim de consultare. Atâta vreme cât există elemente, furnizează în e, următorul element al secvenței f și avansează indicatorul acesteia*
- *DepuneSecvență (f,e) – pentru secvența f deschisă în regim de scriere, copiază valoarea lui e în elementul următor al secvenței f și avansează indicatorul secvenței*

Tipuri structurate

Exemplu de implementare cu fişiere al TDA secvenţă în C

```
FILE * fp;

FILE *fopen(const char *filename, const char *mode)    //
deschidere, atribuire, rescrie/adaugă

size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream)
//furnizează secvenţă

size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE
*stream) //depunde secvenţă
```

Exerciții

Ex1. Descrieți o reprezentare pentru numere întregi care să nu aibă nicio restricție legată de valoarea maximă ce poate fi reprezentată (în afară de restricțiile legate de resursele de memorie ale sistemului de calcul). Descrieți pe scurt cum poate fi folosită această reprezentare pentru calcule de adunare, înmulțire și ridicare la putere.

Ex2. Definiți un TDA listă de întregi. Specificați ce operații doriți să fie definite pentru acest TDA. Apoi propuneți un fișier de tip header pentru o implementare a unei biblioteci în C pentru TDA abstract definit anterior.

Ex3. Implementați un algoritm de sortare generic pentru vectori de elemente de tip structură. Funcția trebuie să poată fi folosită fără a trebui să fie definit în prealabil tipul structură și să poată sorta vectorul indiferent de tipul câmpului cheie după care se va face sortarea (într-un mod asemănător cu funcția qsort).

Bibliografie selectivă

- Crețu, V. Structuri de date și algoritmi, Editura Orizonturi Universitare Timișoara, 2011
- Shaffer, C., “Data Structures and Algorithm Analysis”, 2012